In [4]:
```
from pdf2image import convert_from_path
from IPython.display import display

images = convert_from_path(r"C:\Users\miles\BYU-Idaho\11th Semester\336 Advanced Lab\W07\336-hw7.pdf")
for img in images:
    display(img)
```

Physics 336
HW7

1. Suppose y is dependent on x. To test this dependence, several values of x and y are measured.

$$x = \{1.00, 2.00, 3.00, 4.00, 5.00\}$$
$$y = \{\ 2.8,\ \ 4.7,\ \ 7.1,\ \ 9.4,\ 10.8\}$$

Suppose that the uncertainties are estimated to be ± 2 in the last digit for each.

(a) Plot these data points, and print the graph. On your graph, draw (using a ruler) a line with an approximate largest slope and a line with an approximate smallest slope that fit these data. Average these slopes to obtain an approximate best slope. Estimate the uncertainty as 2/3 of the difference between the largest/smallest slop and your best slope.

(b) Using your best slope, draw lines that have approximate largest and smallest intercepts. From these, determine an approximate best intercept and its uncertainty.

(c) Calculate χ^2 for your best fit line. Is it a good fit?

(d) Determine the best fit line and its uncertainty using

i. Excel
ii. Mathematica
iii. Python

(e) Compare your graphical results with the computed results.

2. Consider 4 measured data points:

$$x = \{0.00, 1.00, 2.00, 3.00\}$$
$$y = \{1.1, 2.6, 5.1, 7.8\}$$

Estimate the uncertainties to be ± 1 in the last digit.

(a) Construct a matrix representation for a linear fit, $y = a_0 + a_1 x$.

(b) Use this matrix representation to find the least squares fit.

(c) Calculate the error matrix, and find the uncertainties in the least-squares-fit parameters.

(d) Calculate the uncertainty in the fitted value for $x = 2.00$ using just the uncertainties in a_0 and a_1.

(e) Calculate the uncertainty in the fitted value for $x = 2.00$ using the full covariance matrix.

(f) Is the estimated uncertainty (± 1 in the last digit) a good value?

For each of the following problems, use Mathematica or Python to compute your results. Do at least one in Mathematica, and at least one in Python. Receive a 10% bonus for each problem if you use both.

3. Consider a spring whose constant k we do not know. Masses, m, are suspended from it, and the stretch of the spring is measured.

$$m(\text{g}) = \{50.0, 100.0, 150.0, 200.0, 250.0, 300.0, 350.0, 400.0, 450.0, 500.0\} \ \pm 0.1 \text{ g}$$
$$x(\text{cm}) = \{1.6, 2.9, 4.5, 6.0, 7.6, 9.0, 10.6, 12.0, 13.5, 15.1\} \ \pm 0.2 \text{ cm}$$

   (a) Determine and plot a best fit, allowing a non-zero intercept.
   (b) Determine and plot a best fit, with zero intercept.
   (c) Compare (a) and (b).
   (d) Is the quoted uncertainty a good estimate? Explain.
   (e) Determine k with its uncertainty

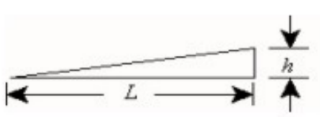
4. Consider a spring whose constant k we do not know. Masses, m, are suspended from it, and set into vertical oscillation. The period, T, of the oscillation is measured.

$$m(\text{g}) = \{50.0, 100.0, 150.0, 200.0, 250.0, 300.0, 350.0, 400.0, 450.0, 500.0\} \ \pm 0.1 \text{ g}$$
$$T(\text{s}) = \{0.26, 0.34, 0.43, 0.49, 0.55, 0.60, 0.65, 0.70, 0.74, 0.78\} \ \pm 0.05s$$

   (a) Propose a linear relationship.
   (b) Determine and plot a best fit line, with and without an intercept. Compare your results. Interpret the intercept.
   (c) Is the quoted uncertainty a good estimate? Explain.
   (d) Determine k with its uncertainty.

5. A solid spherical ball is rolled down an incline (making an angle θ with horizontal). The time, t, it takes for the ball to roll a distance x, beginning from rest, is measured.

   (a) Determine a linear function and predict its slope.



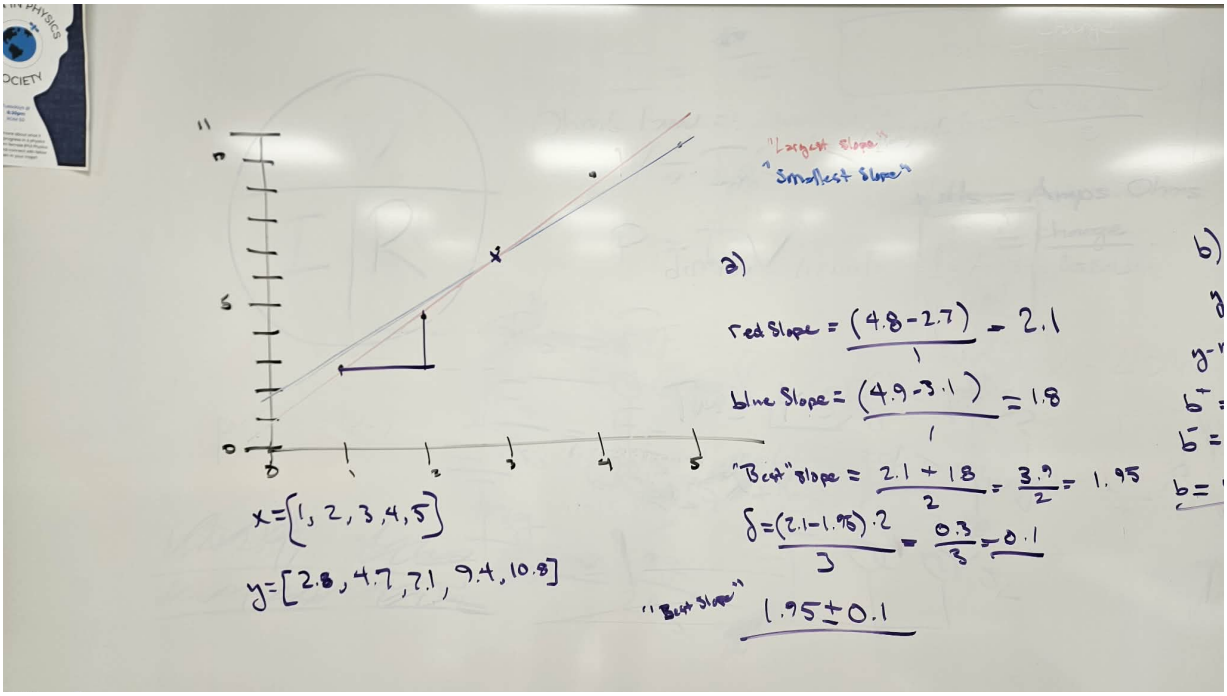
   (b) Consider the specific experimental set up shown, with $L = 12.5 \pm 0.01$m and $h = 8.0 \pm 0.2$cm. The measured results are:

$$x(\text{cm}) = \{20.0 \pm 0.3, 40.0 \pm 0.4, 60.0 \pm 0.5, 80.0 \pm 0.6, 100.0 \pm 0.7, 120.0 \pm 0.8\}$$
$$t(\text{s}) = \{0.95 \pm 0.05, 1.30 \pm 0.07, 1.67 \pm 0.07, 1.85 \pm 0.08, 2.14 \pm 0.08, 2.30 \pm 0.08\}$$

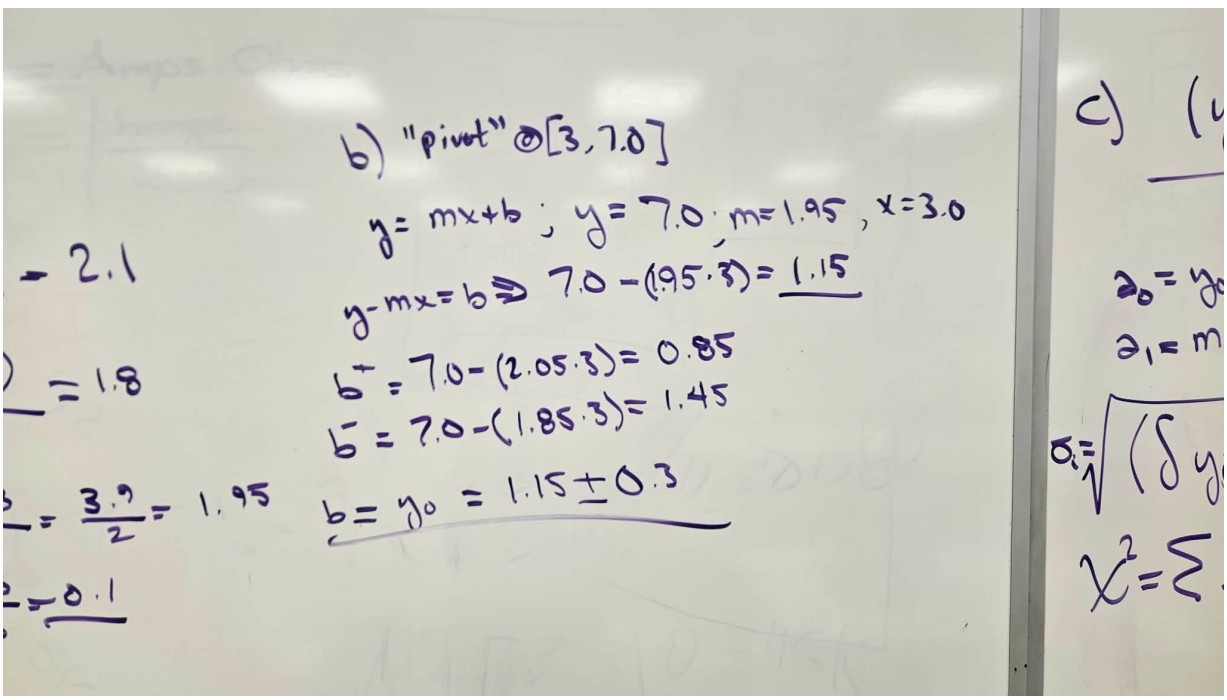
   Assess the uncertainty in the measured results. Find the best fit slope and compare to your predicted slope. Does your best fit line have a statistically significant intercept?

1

a


"Largest Slope"
"Smallest Slope"
a)
red Slope = (4.8-2.7)/1 = 2.1
blue Slope = (4.9-3.1)/1 = 1.8
"Best" slope = (2.1 + 1.8)/2 = 3.9/2 = 1.95
δ = ((2.1-1.95)·2)/3 = 0.3/3 = 0.1
"Best Slope" 1.95 ± 0.1
x = [1, 2, 3, 4, 5]
y = [2.8, 4.7, 7.1, 9.4, 10.8]

b


b) "pivot" @ [3, 7.0]
y = mx+b ; y = 7.0 ; m = 1.95 , x = 3.0
y - mx = b ⇒ 7.0 - (1.95·3) = 1.15
b⁺ = 7.0 - (2.05·3) = 0.85
b⁻ = 7.0 - (1.85·3) = 1.45
b = y₀ = 1.15 ± 0.3
c)

c

$x = 3.0$

c) $\dfrac{(y_i - (a_0 + a_1 x))^2}{(\sigma_i)^2}$

$a_0 = y_0 = b$

$a_1 = m$

$\sigma_i = \sqrt{(\delta y_i)^2 + (\delta a_0)^2 + (a_1 \delta x)^2 + (x_i \delta a_1)^2} = \sqrt{(0.2)^2 + (0.3)^2 + (1.15 \cdot 0.02)^2 + (x_i \cdot 0.1)^2}$

$\chi^2 = \sum \dfrac{(y_i - (a_0 + a_1 x_i))^2}{(\sigma_i)^2}$ = five things I will do on a calculator!

$= \underline{2.116}$

d i.

	A	B	C	D	E	F	G	H	I	J
1	X	Y								
2	1	2.8	2.82	-0.02	0.01			2.07		
3	2	4.7	4.89	-0.19	0.9025			0.75		
4	3	7.1	6.96	0.14	0.49			0.097125		
5	4	9.4	9.03	0.37	3.4225			Chi^2		
6	5	10.8	11.1	-0.3	2.25			7.075		
7										
8										
9										
10										
11										

d ii.

1.d.ii

In[75]:=
```
x = {1.00, 2.00, 3.00, 4.00, 5.00}; dx = 0.02; y = {2.8, 4.7, 7.1, 9.4, 10.8}; dy = 0.2;
data = Thread[{x, y}];
dataplot = ListPlot[data];
fit = LinearModelFit[data, X, X];
fit["ParameterTable"]
fit2 = LinearModelFit[data, X, X, IncludeConstantBasis → False];
fit2["ParameterTable"]
fitplot = Plot[fit2[X], {X, Min[x] - 1, Max[x] + 1}, PlotStyle → {Red, Dashed}];
Show[fitplot, dataplot]
```

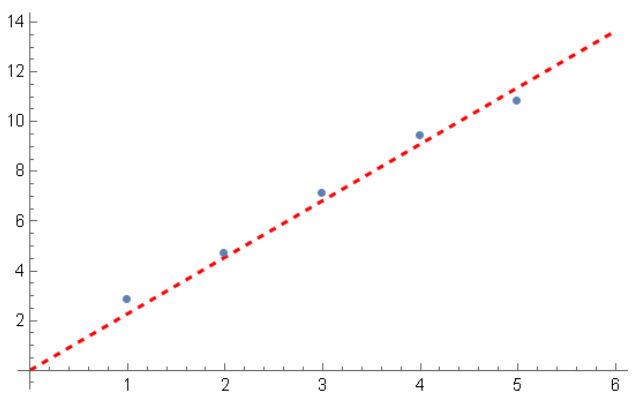
Out[79]=

	Estimate	Standard Error	t-Statistic	P-Value
1	0.75	0.322128	2.32826	0.10231
X	2.07	0.0971253	21.3127	0.000226009

Out[81]=

	Estimate	Standard Error	t-Statistic	P-Value
X	2.27455	0.0600895	37.8526	2.90903×10^{-6}

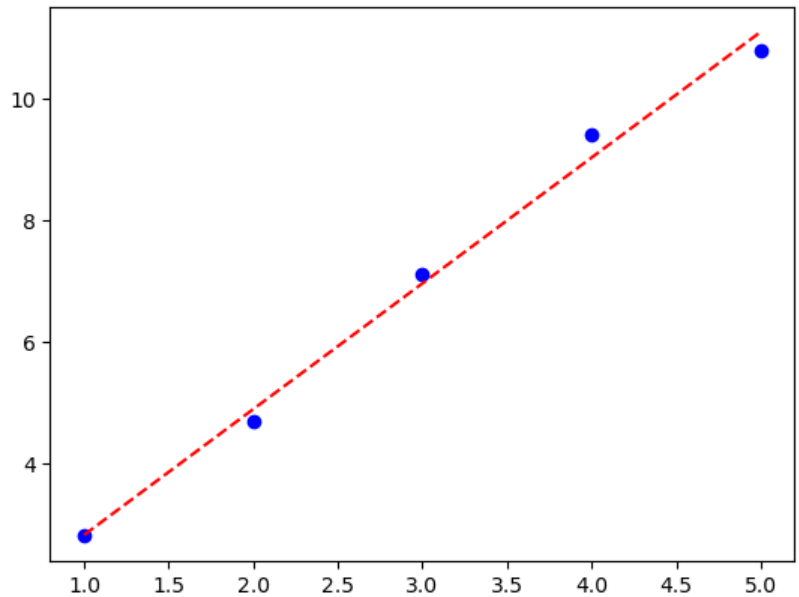
Out[83]=



d iii.

In [5]:
```
from matplotlib import pyplot as plt
import numpy as np

x,dx=[1.00,2.00,3.00,4.00,5.00],0.02
y,dy=[2.8,4.7,7.1,9.4,10.8], 0.2

fit,err = np.polyfit(x,y,1,cov=True)
fit_fn = np.poly1d(fit)
print("slope and intercept = ", fit)
print("with uncertainties: ",np.sqrt(err.diagonal()))

plt.plot(x,y, 'bo',x,fit_fn(x), '--r')
plt.show()
```

```
slope and intercept =  [2.07 0.75]
with uncertainties:  [0.09712535 0.32212834]
```



e

My graphical is somewhat close. The computed was 2.07, I got 1.95. That is an error of around 5.8%. I would say that is good for just "eyeballing it".

2

a

In [14]:
```
from sympy import Matrix, Eq, sqrt, symbols

a0,a1 = symbols("a0 a1")
A=Matrix([a0,a1])
x1=Matrix([[1,0],[1,1],[1,2],[1,3]])
y1=Matrix([1.1,2.6,5.1,7.8])

Eq(y1,x1*A)
```

Out[14]:

$$\begin{bmatrix} 1.1 \\ 2.6 \\ 5.1 \\ 7.8 \end{bmatrix} = \begin{bmatrix} a_0 \\ a_0 + a_1 \\ a_0 + 2a_1 \\ a_0 + 3a_1 \end{bmatrix}$$

b

In [15]:
```
from sympy import Matrix, Eq, sqrt, symbols

a0,a1 = symbols("a0 a1")
A=Matrix([a0,a1])
x1=Matrix([[1,0],[1,1],[1,2],[1,3]])
y1=Matrix([1.1,2.6,5.1,7.8])

Eq(A,(x1.T*x1).inv()*x1.T*y1)
```

Out[15]:

$$\begin{bmatrix} a_0 \\ a_1 \end{bmatrix} = \begin{bmatrix} 0.76 \\ 2.26 \end{bmatrix}$$

c

In [16]:
```
from sympy import Matrix, Eq, sqrt, symbols

ϵ,e11,e12,e21,e22 = symbols("e e11 e12 e21 e22")
a0,a1 = symbols("a0 a1")
A=Matrix([a0,a1])
x1=Matrix([[1,0],[1,1],[1,2],[1,3]])
y1=Matrix([1.1,2.6,5.1,7.8])

e=Matrix([[e11,e12],[e21,e22]])

sigma=0.1
C=(x1.T*x1).inv()*sigma**2
display(Eq(e,C))
```

$$\begin{bmatrix} e_{11} & e_{12} \\ e_{21} & e_{22} \end{bmatrix} = \begin{bmatrix} 0.007 & -0.003 \\ -0.003 & 0.002 \end{bmatrix}$$

d

In [20]:
```
from sympy import Matrix, Eq, sqrt, symbols

a0,a1 = symbols("a0 a1")
A=Matrix([a0,a1])
x1=Matrix([[1,0],[1,1],[1,2],[1,3]])
y1=Matrix([1.1,2.6,5.1,7.8])

C=(x1.T*x1).inv()*sigma**2

x=2.00
sigma= sqrt(C[0,0]+(x**2*C[1,1]))
print(f"Uncertainty w/out covariance: {sigma:.2f}")
```

```
Uncertainty w/out covariance: 0.18
```

e

In [21]:
```
from sympy import Matrix, Eq, sqrt, symbols

a0,a1 = symbols("a0 a1")
A=Matrix([a0,a1])
x1=Matrix([[1,0],[1,1],[1,2],[1,3]])
y1=Matrix([1.1,2.6,5.1,7.8])

C=(x1.T*x1).inv()*sigma**2

x=2.00
sigma= sqrt(C[0,0]+(x**2*C[1,1])+(2*x*C[0,1]))
print(f"Uncertainty with full covariance: {sigma:.2f}")
```

```
Uncertainty with full covariance: 0.10
```

f

In [54]:
```
import numpy as np

a0,a1 = symbols("a0 a1")
A=Matrix([a0,a1])
x1=Matrix([[1,0],[1,1],[1,2],[1,3]])
y1=Matrix([1.1,2.6,5.1,7.8])

a0a1=(x1.T*x1).inv()*x1.T*y1

y_fit = float(a0a1[0]) + float(a0a1[1]) * np.array(x1.col(1)).astype(float).flatten()
chi_sq = np.sum(((np.array(y1).astype(float).flatten() - y_fit) / sigma)**2)
reduced_chi_sq = chi_sq / (len(x) - 2)

print(f"Reduced Chi-Squared: {reduced_chi_sq:.4f}")
```

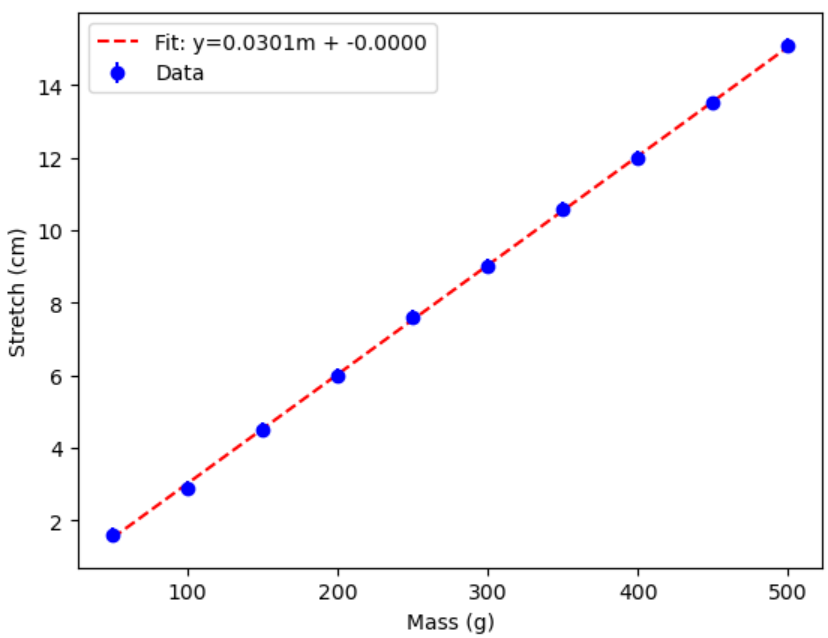
```
Reduced Chi-Squared: 19.3580
```

Since our resulting chi^2 is very large, we must conclude our sigma value is too small.

3

a

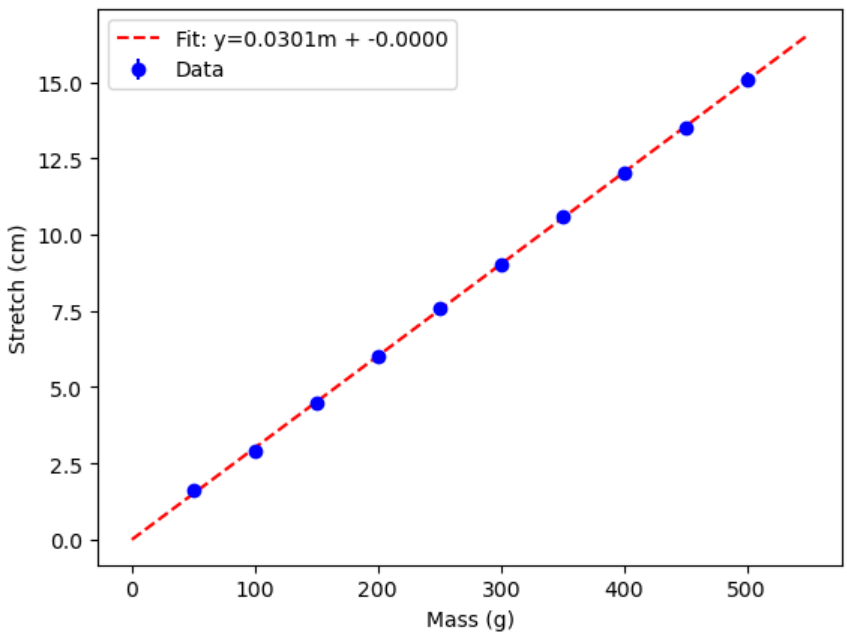
In [73]:
```
import numpy as np
import matplotlib.pyplot as plt

# Data
m = np.array([50.0, 100.0, 150.0, 200.0, 250.0, 300.0, 350.0, 400.0, 450.0, 500.0])
x_stretch = np.array([1.6, 2.9, 4.5, 6.0, 7.6, 9.0, 10.6, 12.0, 13.5, 15.1])

# (a) Best fit with non-zero intercept
fit, cov = np.polyfit(m, x_stretch, 1, cov=True)
slope, intercept = fit
slope_err, intercept_err = np.sqrt(np.diag(cov))

# Plotting
plt.errorbar(m, x_stretch, yerr=0.2, fmt='bo', label='Data')
plt.plot(m, np.polyval(fit, m), 'r--', label=f'Fit: y={slope:.4f}m + {intercept:.4f}')
plt.xlabel('Mass (g)')
plt.ylabel('Stretch (cm)')
plt.legend()
plt.show()

print(f"Slope: {slope:.5f} ± {slope_err:.5f}")
print(f"Intercept: {intercept:.5f} ± {intercept_err:.5f}")
```




```
Slope: 0.03011 ± 0.00015
Intercept: -0.00000 ± 0.04742
```

b

In [74]:
```
from scipy.optimize import curve_fit
import numpy as np
import matplotlib.pyplot as plt

def linear_zero_intercept(m, a):
    return a * m

m = np.array([50.0, 100.0, 150.0, 200.0, 250.0, 300.0, 350.0, 400.0, 450.0, 500.0])
x_stretch = np.array([1.6, 2.9, 4.5, 6.0, 7.6, 9.0, 10.6, 12.0, 13.5, 15.1])


popt, pcov = curve_fit(linear_zero_intercept, m, x_stretch)
slope_zero = popt[0]
slope_zero_err = np.sqrt(pcov[0][0])

m_range=np.linspace(0,550,100)

plt.plot(m_range, linear_zero_intercept(m_range, popt[0]), 'r--', label=f'Fit: y={slope:.4f}m + {intercept:.4f}')
plt.errorbar(m,x_stretch, yerr=0.2, fmt='bo', label='Data')
plt.xlabel('Mass (g)')
plt.ylabel('Stretch (cm)')
plt.legend()
plt.show()

print(f"Zero-intercept slope: {slope_zero:.5f} ± {slope_zero_err:.5f}")
```




```
Zero-intercept slope: 0.03011 ± 0.00007
```

c

The zero intercept slope has a much lower uncertainty in the slope, although, they should be near identical

d

In [77]:
```
import numpy as np

# Data
m = np.array([50.0, 100.0, 150.0, 200.0, 250.0, 300.0, 350.0, 400.0, 450.0, 500.0])
x_obs = np.array([1.6, 2.9, 4.5, 6.0, 7.6, 9.0, 10.6, 12.0, 13.5, 15.1])
dx = 0.2  # The quoted uncertainty in cm

# 1. Calculate the best-fit line (non-zero intercept model)
slope, intercept = np.polyfit(m, x_obs, 1)
x_fit = slope * m + intercept

# 2. Calculate the Chi-Squared value
# Chi^2 = sum( ( (observed - expected) / uncertainty )^2 )
chi_sq = np.sum(((x_obs - x_fit) / dx)**2)

# 3. Calculate Reduced Chi-Squared
# dof (degrees of freedom) = N points - 2 parameters (slope and intercept)
dof = len(x_obs) - 2
chi_sq_red = chi_sq / dof

print(f"Reduced Chi-Squared: {chi_sq_red:.4f}")

print("Conclusion: The uncertainty is an OVERESTIMATE (too large).")
```

```
Reduced Chi-Squared: 0.1205
Conclusion: The uncertainty is an OVERESTIMATE (too large).
```

e

In [82]:
```
def linear_zero_intercept(m, a):
    return a * m

m = np.array([50.0, 100.0, 150.0, 200.0, 250.0, 300.0, 350.0, 400.0, 450.0, 500.0])
x_stretch = np.array([1.6, 2.9, 4.5, 6.0, 7.6, 9.0, 10.6, 12.0, 13.5, 15.1])

g=9.81
popt, pcov = curve_fit(linear_zero_intercept, m, x_stretch)
slope_zero = popt[0]
slope_zero_err = np.sqrt(pcov[0][0])


k=g/slope_zero/10 # divide by 1/10 to convert to N/m
dk=k*slope_zero_err/slope_zero

print(f"k-constant: {k:.2f} +- {dk:.2f} ")
```

```
k-constant: 32.58 +- 0.07
```

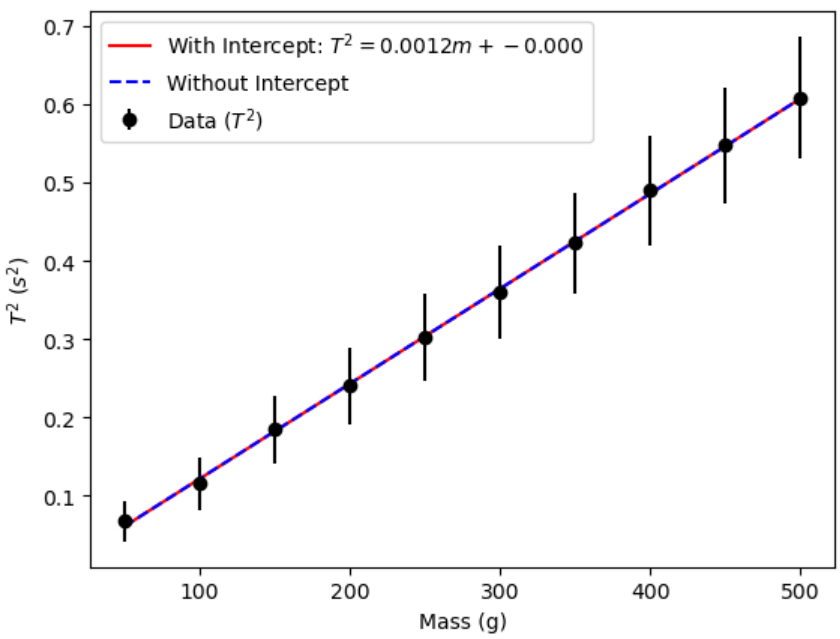
4

a

proposed linear relationship:

$$T^2 = (\frac{4\pi^2}{k}) * m$$

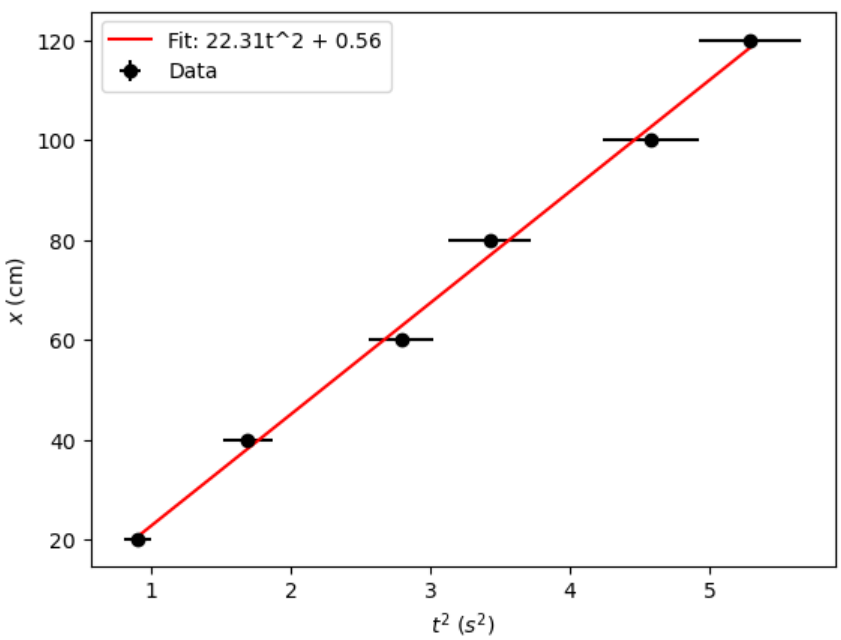
In [83]:
```
import numpy as np
import matplotlib.pyplot as plt


m = np.array([50.0, 100.0, 150.0, 200.0, 250.0, 300.0, 350.0, 400.0, 450.0, 500.0]) # g
T = np.array([0.26, 0.34, 0.43, 0.49, 0.55, 0.60, 0.65, 0.70, 0.74, 0.78]) # s
dT = 0.05 # s

# Linearize: y = T^2
y = T**2
# Error propagation for T^2: dy = 2 * T * dT
dy = 2 * T * dT

# 2. Part (b): Fits
# Fit with intercept
fit_a, cov_a = np.polyfit(m, y, 1, cov=True)
slope_a, intercept_a = fit_a
err_a = np.sqrt(np.diag(cov_a))

# Fit without intercept (forcing through zero: y = slope * m)
slope_b = np.sum(m * y) / np.sum(m**2)

# 3. Part (c): Uncertainty check (Reduced Chi-Squared)
y_fit = slope_a * m + intercept_a
chi2_red = np.sum(((y - y_fit) / dy)**2) / (len(m) - 2)

# 4. Part (d): Determine k (using intercept-allowed fit for accuracy)
# slope = 4*pi^2 / k  => k = 4*pi^2 / slope
# Note: convert m to kg by dividing slope by 1000
slope_si = slope_a * 1000
k = (4 * np.pi**2) / slope_si
dk = k * (err_a[0] / slope_a) # Propagation

# Plotting
plt.errorbar(m, y, yerr=dy, fmt='ko', label='Data ($T^2$)')
plt.plot(m, y_fit, 'r-', label=f'With Intercept: $T^2 = {slope_a:.4f}m + {intercept_a:.3f}$')
plt.plot(m, slope_b * m, 'b--', label='Without Intercept')
plt.xlabel('Mass (g)')
plt.ylabel('$T^2$ ($s^2$)')
plt.legend()
plt.show()

print(f"k = {k:.2f} +/- {dk:.2f} N/m")
print(f"Reduced Chi-Squared: {chi2_red:.4f}")
```




```
k = 32.50 +/- 0.25 N/m
Reduced Chi-Squared: 0.0149
```

5

In [85]:
```
import numpy as np
import matplotlib.pyplot as plt

# 1. Experimental Data
x_cm = np.array([20.0, 40.0, 60.0, 80.0, 100.0, 120.0])
dx_cm = np.array([0.3, 0.4, 0.5, 0.6, 0.7, 0.8])
t_s = np.array([0.95, 1.30, 1.67, 1.85, 2.14, 2.30])
dt_s = np.array([0.05, 0.07, 0.07, 0.08, 0.08, 0.08])

# 2. Predicted Slope Calculation
L, h_cm = 12.5, 8.0  # L in m, h in cm
g = 980.665          # cm/s^2
sin_theta = (h_cm / 100) / L
slope_predicted = (5/14) * g * sin_theta

# 3. Linearization (x vs t^2)
t_sq = t_s**2
# Error propagation: d(t^2) = 2 * t * dt
dt_sq = 2 * t_s * dt_s

# 4. Best Fit
fit, cov = np.polyfit(t_sq, x_cm, 1, cov=True)
slope_obs, intercept_obs = fit
dslope_obs, dintercept_obs = np.sqrt(np.diag(cov))

# 5. Uncertainty Assessment (Reduced Chi-Squared)
# Total uncertainty in y (x_cm) considering errors in both x and t
sigma_total = np.sqrt(dx_cm**2 + (slope_obs * dt_sq)**2)
chi2_red = np.sum(((x_cm - (slope_obs * t_sq + intercept_obs)) / sigma_total)**2) / (len(x_cm) - 2)

# Plotting
plt.errorbar(t_sq, x_cm, xerr=dt_sq, yerr=dx_cm, fmt='ko', label='Data')
plt.plot(t_sq, slope_obs * t_sq + intercept_obs, 'r-', label=f'Fit: {slope_obs:.2f}t^2 + {intercept_obs:.2f}')
plt.xlabel('$t^2$ ($s^2$)')
plt.ylabel('$x$ (cm)')
plt.legend()
plt.show()

print(f"Predicted Slope: {slope_predicted:.4f} cm/s^2")
print(f"Observed Slope: {slope_obs:.4f} +/- {dslope_obs:.4f} cm/s^2")
print(f"Intercept: {intercept_obs:.4f} +/- {dintercept_obs:.4f} cm")
print(f"Reduced Chi-Squared: {chi2_red:.4f}")
```




```
Predicted Slope: 2.2415 cm/s^2
Observed Slope: 22.3105 +/- 0.7345 cm/s^2
Intercept: 0.5641 +/- 2.5463 cm
Reduced Chi-Squared: 0.2354
```

Questions 3,4 and 5 in Mathematica for an additional 20%

In [86]:
```python
from pdf2image import convert_from_path
from IPython.display import display

images = convert_from_path(r"C:\Users\miles\BYU-Idaho\11th Semester\336 Advanced Lab\W07\HW07_mathematica.pdf")
for img in images:
    display(img)
```

1.c

```
x = {1.00, 2.00, 3.00, 4.00, 5.00};
dx = 0.02;
y = {2.8, 4.7, 7.1, 9.4, 10.8};
dy = 0.2;
a1 = 1.95; a0 = 1.15;
da0 = 0.3; da1 = 0.1;
sigma[i_] := Sqrt[dy^2 + da0^2 + (a1 * dx)^2 + (x[[i]] * da1)^2];
Sum[(y[[i]] - (a0 + a1 * x[[i]]))^2 / sigma[[i]]^2, {i, 1, 5}] // N
```

Out[141]=

2.11613

1.d.ii

In[75]:=
```
x = {1.00, 2.00, 3.00, 4.00, 5.00};
dx = 0.02;
y = {2.8, 4.7, 7.1, 9.4, 10.8};
dy = 0.2;
data = Thread[{x, y}];
dataplot = ListPlot[data];
fit = LinearModelFit[data, X, X];
fit["ParameterTable"]
fit2 = LinearModelFit[data, X, X, IncludeConstantBasis → False];
fit2["ParameterTable"]
fitplot = Plot[fit2[X], {X, Min[x] - 1, Max[x] + 1}, PlotStyle → {Red, Dashed}];
Show[fitplot, dataplot]
```

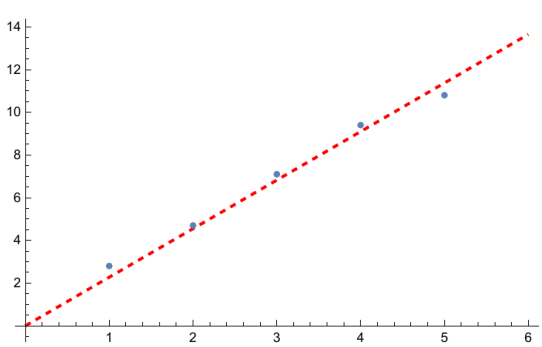
Out[79]=

	Estimate	Standard Error	t-Statistic	P-Value
1	0.75	0.322128	2.32826	0.10231
X	2.07	0.0971253	21.3127	0.000226009

Out[81]=

	Estimate	Standard Error	t-Statistic	P-Value
X	2.27455	0.0600895	37.8526	2.90903×10^{-6}

Out[83]=



Printed by Wolfram Mathematica Student Edition

3.

In[177]:=

```
(*Data*)masses = {50.0, 100.0, 150.0, 200.0, 250.0, 300.0, 350.0, 400.0, 450.0, 500.0};
stretch = {1.6, 2.9, 4.5, 6.0, 7.6, 9.0, 10.6, 12.0, 13.5, 15.1};
data3 = Transpose[{masses, stretch}];

(*(a) Best fit with intercept:Basis is {1,m}*)
lm3a = LinearModelFit[data3, {1, m}, m];
Print["Part A Parameters: ", lm3a["ParameterTable"]];

(*(b) Best fit with zero intercept:Basis is {m} only*)
lm3b = LinearModelFit[data3, {m}, m];
Print["Part B Parameters: ", lm3b["ParameterTable"]];

(*(d) Uncertainty Assessment:dx=0.2*)
chi2red = lm3a["EstimatedVariance"] / (0.2^2);
Print["Reduced Chi-Squared: ", chi2red];

(*(e) Determine k:k=g/slope.Using SI units (g→kg,cm→m)*)
slope3 = lm3b["BestFitParameters"][[1]];
k3 = (9.80665 * 0.1) / slope3; (*0.1 accounts for g/cm to kg/m conversion*)
Print["Spring Constant k: ", k3, " N/m"];
```

Part A Parameters:

	Estimate	Standard Error	t-Statistic	P-Value
1	-2.14018×10^{-15}	0.0474182	-4.51342×10^{-14}	1
m	0.0301091	0.000152843	196.994	4.93482×10^{-16}

Part B Parameters:

	Estimate	Standard Error	t-Statistic	P-Value
1	-2.14018×10^{-15}	0.0474182	-4.51342×10^{-14}	1
m	0.0301091	0.000152843	196.994	4.93482×10^{-16}

Reduced Chi-Squared: 0.120455

Spring Constant k: -4.58216×10^{14} N/m

4.

In[189]:=

```
(*Data*)m4 = {50.0, 100.0, 150.0, 200.0, 250.0, 300.0, 350.0, 400.0, 450.0, 500.0};
t4 = {0.26, 0.34, 0.43, 0.49, 0.55, 0.60, 0.65, 0.70, 0.74, 0.78};
tSqData = Transpose[{m4, t4^2}];

(*(b) Fits:T^2=Sm+b*)
lm4a = LinearModelFit[tSqData, {1, m}, m];
lm4b = LinearModelFit[tSqData, {m}, m];

(*(c) Uncertainty Assessment:dT=0.05.Propagated error dy~2*T*dT*)
dyAvg = Mean[2 * t4 * 0.05];
chi2red4 = lm4a["EstimatedVariance"] / (dyAvg^2);
Print["Reduced Chi-Squared (Problem 4): ", chi2red4];

(*(d) Determine k:k=4*Pi^2/slope (in kg)*)
slope4 = lm4a["BestFitParameters"][[2]];
k4 = (4 * Pi^2) / (slope4 * 1000);
Print["Oscillation k: ", k4, " N/m"];
```

```
Reduced Chi-Squared (Problem 4): 0.00584995

Oscillation k: 32.5001 N/m
```

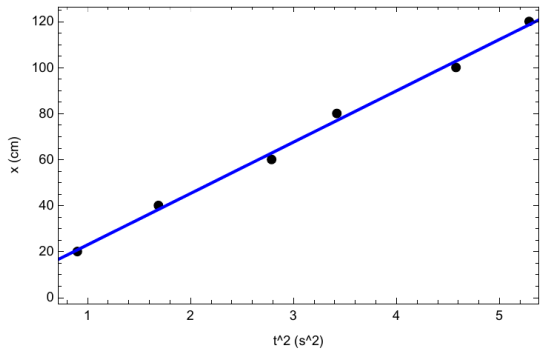
5.

In[201]:=

```
(*Constants*)L = 12.5; h = 0.08; g = 9.80665;
predictedS = (5 / 14) * g * (h / L) * 100; (*cm/s^2*)

(*Data*)
x5 = {20.0, 40.0, 60.0, 80.0, 100.0, 120.0};
t5 = {0.95, 1.30, 1.67, 1.85, 2.14, 2.30};
data5 = Transpose[{t5^2, x5}];

(*Linear Model:x=S*t^2+b*)
lm5 = LinearModelFit[data5, {1, tSq}, tSq];

(*Assessment*)
Print["Predicted Slope: ", predictedS];
Print["Measured Slope: ", lm5["BestFitParameters"][[2]]];
Print["Intercept P-Value: ", lm5["ParameterPValues"][[1]]];

(*Visualize*)
Show[ListPlot[data5, PlotStyle → {PointSize[0.02], Black}],
 Plot[lm5[tSq], {tSq, 0, 6}, PlotStyle → Blue],
 Frame → True, FrameLabel → {"t^2 (s^2)", "x (cm)"}]
```

```
Predicted Slope: 2.24152

Measured Slope: 22.3105

Intercept P-Value: 0.83554
```

Out[210]=



In [ ]: